

Flask Online Shop

The Brief

A short, accurate tour of a Flask + Stripe storefront

1 What it is

A small web store that sells planets as a joke: nine product cards, accounts with hashed passwords, and a *Buy* button that hands the visitor to Stripe's own payment page. It is about 360 lines of Python in a single file, five HTML templates, one stylesheet, and five pinned dependencies. Its purpose is to wire up the parts a real store needs without a framework that hides them.

The honest summary, up front

The project gets the two genuinely dangerous things right: passwords are bcrypt-hashed and never stored in plain text, and card numbers never touch this application because Stripe hosts the payment form. It gets login state badly wrong: "logged in" is a flag on a database row rather than a property of a visitor's session, so **one person logging in makes every visitor appear logged in**. I verified this against the running app. The defect is severe, localised to six lines, and the most interesting thing here to be able to explain.

2 Architecture at a glance

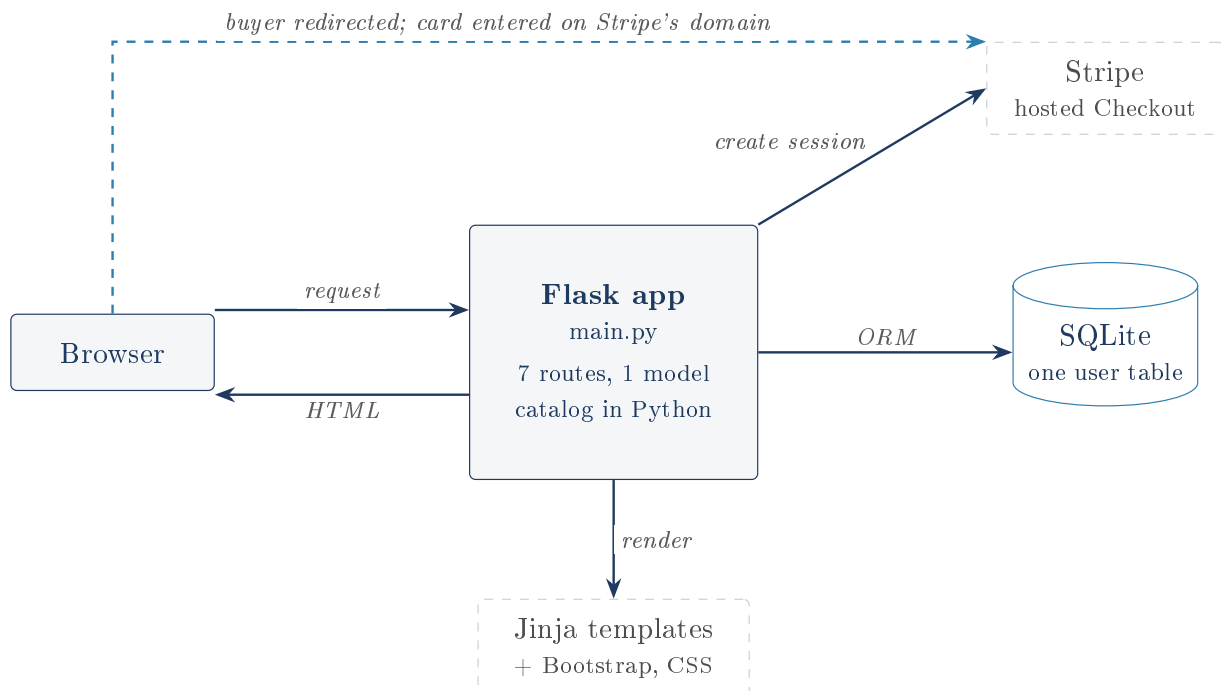


Figure 1: Solid arrows are this app's traffic. The dashed arrow is the buyer leaving for Stripe, and it is why no card number ever enters this codebase.

Piece	Tool	Role
Web framework	Flask 3.0.3	Routing, templates, redirects. Small by design.
Persistence	SQLAlchemy 2.0.30	One <code>User</code> table in a SQLite file under <code>instance/</code> .
Passwords	bcrypt 4.1.3	Salted, deliberately slow hashing.
Payments	stripe 9.12.0	One API call: create a Checkout Session, redirect to it.
Front end	Jinja + Bootstrap 5	One <code>base.html</code> shell; nine cards from one looped block.

3 How a purchase flows

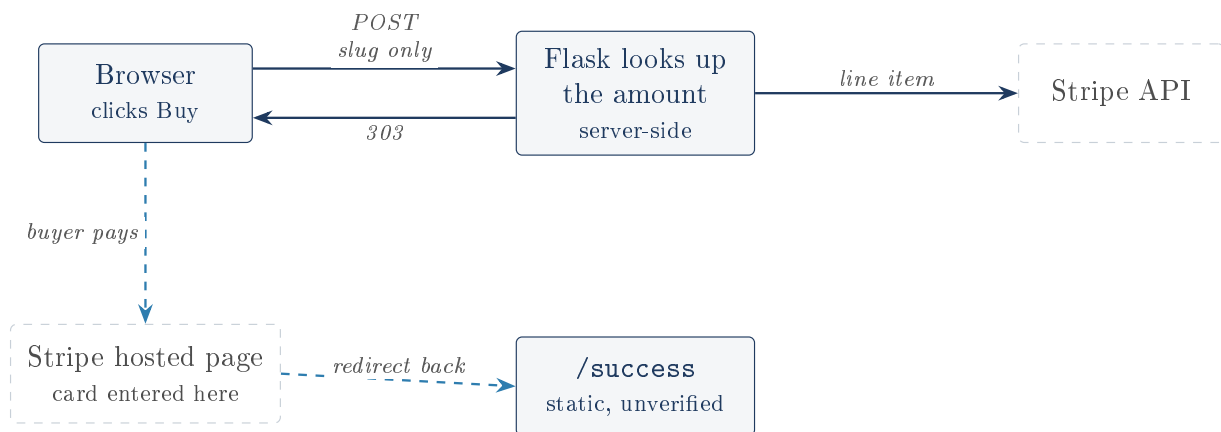


Figure 2: The client sends only a slug such as `earth`; the server owns the price, so a buyer cannot tamper with the amount. But the return path to `/success` is a plain browser redirect, which the buyer fully controls.

One detail worth keeping: the displayed price and the price charged both derive from a single `unit_amount` in cents. The README records that a template once hardcoded "\$69,420" for Uranus while Stripe was told \$42,069. Deriving one from the other makes that bug unrepresentable, and I confirmed all nine prices now agree with their cent amounts.

4 The defect worth understanding

```

1 def is_logged_in():
2     return bool(db.session.query(exists().where(User.logged_in == 1)).scalar())

```

Listing 1: The whole of the login-state logic.

That query asks whether *any* row in the whole table has `logged_in == 1`. It never learns who is asking: no cookie is read, no session consulted, no identity involved. A context processor feeds its answer to the nav bar on every page.

Verified: one login logs everybody in

I drove the running app with three independent HTTP clients, each with its own cookie jar. Before anyone logs in, all three see "Login." After Alice logs in, all three see "Logout," including a brand-new client holding zero cookies that never authenticated.

The README calls this "no real session isolation between browser tabs or devices," which understates it. Tabs and devices are the symptom. The disease is that login is a property of the *application*, not of a visitor. Related, and also confirmed: `/logout` requires no authentication and clears whichever flagged row comes back first, so a stranger can log Alice out;

and because the flag lives in the database it survives restarts.

The fix is small: put the user id in Flask's `session` (a signed cookie, per-visitor by construction) and delete the `logged_in` column. Flask's `session` is already imported and used once, only to clear stale flash messages. Because the surrounding structure is right, changing those few lines fixes every page without touching a template.

5 Security posture, stated plainly

Area	Verdict	Detail
Password storage	Correct	bcrypt with per-user salt; plain text never stored.
Card data	Correct	Never touches this app; Stripe hosts the form.
Price integrity	Correct	Client sends a slug; the server owns the amount.
Secrets	Mostly correct	Read from the environment, <code>.env</code> gitignored. But the signing key falls back to a publicly known default instead of failing loudly.
Session handling	Broken	Global flag, not a session. Verified above.
Payment confirmation	Broken	<code>/success</code> returns 200 to someone who never paid. No webhook, and no orders table to record a payment in even if there were one.
Auth on checkout	Absent	Buying requires no login at all; the account system is decorative with respect to purchasing.
Input validation	Weak	Client-side only. A 60-character password bypasses the form's <code>maxlength="10"</code> ; an unknown planet slug or a missing password field each return HTTP 500.
Data model	Weak	<code>email</code> has no uniqueness constraint (duplicates orphan the second account); the password column is declared <code>String</code> but stores bytes, forcing an <code>isinstance</code> guard at login.
Tests	Absent	No test suite.

The setup instructions do not work as written

The README says to `cp .env.example .env` and edit it. Nothing in this project ever reads a `.env` file: there is no `python-dotenv` dependency and no `load_dotenv()` call anywhere in the tree. A reader who follows the instructions gets the insecure fallback signing key and an empty Stripe key, and finds out only when checkout fails. The variables must be exported into the shell instead. Relatedly, `app.config['SESSION_TYPE'] = 'filesystem'` is a Flask-Session setting, and Flask-Session is neither installed nor imported, so that line does nothing at all.

6 What was and was not verified

I installed the pinned dependencies into a clean virtual environment and drove the running server. **Confirmed working:** it boots; `GET /` returns 200 with exactly nine cards; all nine prices match their Stripe amounts; register, login, wrong password, unknown email and logout all behave as documented; and the per-user password check (which the README says was once hardcoded to compare against user 1) is genuinely fixed, tested with two accounts.

Confirmed broken: everything in the posture table marked broken, weak or absent.

Not verified: a real Stripe payment end to end, which needs a test-mode key this environment does not have. Without a key the checkout route reaches Stripe and fails only on authentication, which proves the wiring up to that boundary and no further. **Inferred, not observed:** Pluto's

`unit_amount` of 9999999999999999 cents (rendered as \$9,999,999,999,999.99) exceeds Stripe's documented USD maximum of 99999999 cents, so its Buy button would be expected to fail while the other eight succeed. The amount is read from the source; the rejection is deduced from Stripe's published limit rather than seen.

7 Verdict

Where the value is

This is a competent beginner web app that is right about the two things that genuinely hurt when they are wrong (plain-text passwords, handling card numbers) and wrong about state management in a textbook way. Its worth in an interview is not "I built a shop." It is being able to point at a global mutable flag standing in for a session, explain exactly why it collapses the moment a second visitor arrives, demonstrate it with three cookie jars, and name the six-line fix; and to say that "redirect to Stripe, then trust the redirect back" is the classic payment mistake, whose real fix is a webhook plus an orders table. Flaws named first by the author read as engineering judgment. The same flaws found first by an interviewer do not.